

LLD Practice Platform

Research Note — Learner Problem & Product Direction

1. The Learner Problem

Low-Level Design (LLD) practice is common in interview preparation — problems like Parking Lot, Elevator, and Vending Machine — but it is difficult for a learner to self-assess. Unlike algorithmic problems, LLD rarely has one correct answer: two structurally different designs can both be valid, which leaves a solo learner unsure whether their classes, responsibilities, and abstractions are actually sound, or just different from an example they saw once.

In practice, this shows up as a few recurring struggles: learners don't know what they're expected to produce (prose, code, or a diagram); they have no reliable way to tell if their responsibility split is reasonable; and without feedback tied to specific evidence in their own submission, an attempt is a one-shot exercise rather than something they can improve on. Nothing closes the loop back to "here is what to fix next time."

2. Existing Approaches Researched

A short review (~2–3 hours) of how learners currently practice LLD surfaced four recurring patterns:

Approach	What it offers	Where it falls short
Static problem banks / write-ups (e.g. curated GitHub LLD problem sets, "Grokking the Low Level Design"-style guides)	Strong problem statements, often with a reference solution to compare against.	No personalized feedback — the learner must manually judge their own solution against a fixed reference.
LeetCode-style OOD / "design"-tagged problems	Familiar structured problem format; large existing audience.	Feedback loop is built for code correctness via test cases, which doesn't fit open-ended design quality.
Mock interview platforms (e.g. Pramp, interviewing.io)	Real, evidence-based human feedback; high quality when available.	Not self-serve — requires scheduling a live person, so it doesn't support quick, repeated solo practice.
Ad-hoc use of general AI chat tools (e.g. asking ChatGPT or Claude "is this a good design?")	On-demand and already widely used by learners informally.	Unconstrained prompting gives inconsistent feedback with no fixed rubric and no persistent history.

3. Key Gaps Identified

- No existing option combines a clear problem statement, structured evidence-based evaluation, and persistent attempt history in a single self-serve, on-demand loop.
- Reference-solution approaches implicitly suggest a single correct answer, which discourages learners whose valid design simply looks different.
- AI-assisted review already happens informally, but without a fixed rubric or structured output it produces inconsistent, non-comparable feedback across attempts.

- Nothing tracks recurring weaknesses across multiple attempts, so the practice loop stays one-shot rather than genuinely improving the learner over time.

4. Product Direction

The platform focuses on a narrow, repeatable practice loop: choose a problem, submit a text-based design, receive rubric-based structured feedback (score, evidence, concern, and suggestion per criterion — not one opaque score), and review attempt history to see whether weaknesses recur.

Evaluation deliberately separates deterministic checks (completeness, valid state transitions) from AI-judged checks (responsibility quality, trade-offs, extensibility, testability), so feedback stays consistent and explainable rather than depending on an unconstrained "is this good?" prompt.

The MVP is scoped to three well-known problems — Parking Lot, Elevator, and Vending Machine — with text-only submissions. This is enough to prove the full evaluation-and-history loop end-to-end, while the domain model (a typed submission format, and evaluation behind a swappable interface) is designed to extend to diagram submissions or additional evaluators later without reworking the core flow.